# Ders 6 Genişletilmiş Notu: Fonksiyonlar, Ters Fonksiyon, Taban ve Tavan, Bölüm Soru Çözümü ve Algoritmalara Giriş: Arama, Sıralama, Öklid

## Ders 6 Genişletilmiş Notu: Fonksiyonlar, Ters Fonksiyon, Taban ve Tavan, Bölüm Soru Çözümü ve Algoritmalara Giriş: Arama, Sıralama, Öklid

> Bu not, ders kaydının altyazısından hazırlanmış ayrıntılı ders notudur. Kısa tekrar için aynı dersin kısa notuna bakın.

### İçindekiler


- 1. Warshall Algoritmasının Tekrarı
- 2. Matris Bağıntıları
- 3. Fonksiyon Kavramı
- 4. Birebir, Örten ve Birebir-Örten Fonksiyonlar
- 5. Ters Fonksiyon
- 6. Fonksiyon Bileşkesi
- 7. Dizginin Tersi
- 8. Taban (Floor) ve Tavan (Ceiling) Fonksiyonları
- 9. Bölüm Soruları: Küme İşlemleri ve Venn Şeması
- 10. Bölüm Soruları: Üç Kümeli Sayma Problemi
- 11. Bölüm Soruları: Doğru/Yanlış ve Karşı Örnek
- 12. Bölüm Soruları: Dizi Toplamları ve Sayı Tabanları
- 13. Bölüm Soruları: Bağıntılar ve Ters Fonksiyon
- 14. Algoritmalar Bölümüne Giriş
- 15. En Büyük Elemanı Bulma ve Sözde Kod
- 16. Bir Algoritmanın Özellikleri
- 17. Problem Türleri ve Doğrusal Arama
- 18. İkili Arama
- 19. Sıralama ve Kabarcık Sıralaması
- 20. Araya Ekleme Sıralaması
- 21. Öklid Algoritması
- 22. Algoritma Tasarım Yöntemleri
- Örnek Sorular
- Dersin Toparlaması


## 1. Warshall Algoritmasının Tekrarı

Ders Zoom üzerinden yapıldı. Kaydın ilk dakikaları bekleme ve selamlaşmadır; asıl anlatım yaklaşık 00:08:40’ta başlar. Hoca, bu hafta fonksiyonları bitirip bölümün soru-cevap slaytlarına bakacaklarını, ardından yeni eklenen algoritmalar bölümüne geçeceklerini söyledi. Bir öğrenci, geçen hafta internet kesildiği için Warshall algoritmasının anlaşılmadığını hatırlattı. Hoca da fonksiyonlara geçmeden önce algoritmayı tekrar anlattı.

### 1.1 Kapanışları Elle Bulmak

Hoca önce dört elemanlı $(a, b, c, d)$ bir bağıntı üzerinden kapanış (closure) fikrini hatırlattı. Bağıntı şudur:

$$R = \{(b, a), (b, c), (c, a), (c, d), (d, c), (a, d)\}$$

- **Yansımalı (reflexive) kapanış:** Yansımalı olmak için bütün $(x, x)$ ikilileri bulunmalıdır. Bunu elle yapmak çok kolaydır: $(a, a), (b, b), (c, c), (d, d)$ eklenir.
- **Simetrik kapanış:** Her $(x, y)$ için $(y, x)$ aranır. $(b, a)$ var, $(a, b)$ yok, eklenir. $(b, c)$ için $(c, b)$, $(c, a)$ için $(a, c)$ eklenir. $(c, d)$ için $(d, c)$ zaten vardır. $(a, d)$ için $(d, a)$ eklenir. Bu da kolaydır.
- **Geçişli (transitive) kapanış:** Asıl zor olan budur. Bağıntının geçişli olmadığını göstermek için tek bir örnek yeter. Örneğin $(b, a)$ ve $(a, d)$ var; ortadaki $a$'lar ortak olduğu için $(b, d)$ de olmalıdır, ama yoktur. Elle $(b, d)$ eklenir ve bu kontrol bütün ikililer için tekrarlanır.

Hoca, elinizde çok sayıda ikili olduğunda bunu gözle yapmanın bile zor olduğunu, bu yüzden bir algoritmaya ihtiyaç duyulduğunu söyledi. Algoritmanın hiçbir eksik bırakmaması ve aynı zamanda hızlı çalışması gerekir. Hocanın anlattığına göre akademide bunun için bir yarışma açılmış. Herkes kendi algoritmasını yarıştırmış; kaç düğümlü bir graf verilirse verilsin, geçişli olmayan bir bağıntıyı en hızlı şekilde geçişli yapan algoritma Warshall'ın algoritması olmuş.

> **Ders sonrası not:** Hoca bu kısımda $R$'den birkaç kez "fonksiyon" diye söz etti. $R$ bir bağıntıdır (relation), fonksiyon değildir: $b$ hem $a$'ya hem $c$'ye gidiyor.

### 1.2 Graftan ve Listeden Matrise

Hocaya göre matematikte bir problemi çözmek istiyorsak onu ya bir dizi ya da bir matris kıvamına indirmeliyiz. Düğümler arasındaki geçişleri veren bir grafı anlatmanın en güzel yolu matristir.

Bağıntı liste olarak verildiği gibi graf olarak da çizilebilir. Her $(x, y)$ ikilisi $x$'ten $y$'ye yönlü bir oktur: $b \to a$, $b \to c$, $c \to a$, $c \to d$, $d \to c$, $a \to d$.

Matrise geçmek için illa graf çizmek gerekmez; listeden de geçilir. Satırlar tanım kümesinin (domain), sütunlar değer kümesinin (range) elemanlarıdır. $(x, y) \in R$ ise ilgili göze 1, değilse 0 yazılır. Hoca bu matrise, algoritma Warshall'a ait olduğu için $W$ adını verdi:

| | $a$ | $b$ | $c$ | $d$ |
|---|---|---|---|---|
| $a$ | 0 | 0 | 0 | 1 |
| $b$ | 1 | 0 | 1 | 0 |
| $c$ | 1 | 0 | 0 | 1 |
| $d$ | 0 | 0 | 1 | 0 |

### 1.3 Algoritmanın Çalışması

Warshall algoritması başlangıç matrisini ($R_0$) $W$'ye atar ve üç iç içe döngü kullanır. Hoca neden üç döngü olduğunu şöyle açıkladı: Geçişlilikte kontrol edilen ifade $(x, y)$, $(y, z)$ ve $(x, z)$ olmak üzere üç parçalıdır, bu yüzden üç iç içe döngü gerekir. En dıştaki döngü $k = 1, \ldots, n$'dir; burada $n = 4$'tür.

```
W := M_R
for k := 1 to n
    for i := 1 to n
        for j := 1 to n
            w_ij := w_ij OR (w_ik AND w_kj)
return W
```

Hocanın açıklamasıyla: $w_{ik}$ ve $w_{kj}$ birlikte 1 ise $i \to k \to j$ zinciri vardır ve geçişlilik için $i \to j$ de olmalıdır. VE (AND) işlemi bu zinciri yakalar. VEYA (OR) işlemi ise eski değeri korur:

$w_{ij}$ zaten 1 ise 1 kalır; 0 ise ve zincir varsa 1 yapılır. Kısacası hocanın gözle yaptığı kontrolü algoritma mantık işlemleriyle yapar.

Hoca $k = 1$, $i = 1$ için iki adımı tek tek gösterdi:

- $j = 1$: $w_{11} = w_{11} \vee (w_{11} \wedge w_{11}) = 0 \vee (0 \wedge 0) = 0$. Yani $(a, a)$ eklenmez.
- $j = 4$: $w_{14} = w_{14} \vee (w_{11} \wedge w_{14}) = 1 \vee (0 \wedge 1) = 1$. Burada $(a, a)$ ve $(a, d)$ "içleri aynı" olan ikililerdir; dışarıda $(a, d)$ olmalıdır. $(a, d)$ zaten var olduğu için değer 1 kalır, değişmez.

Ara matrisler derste gösterilmedi; Python ile hesaplandı (ara adım eklendi). Her $k$ adımından sonra değişen gözler:

| Adım | Eklenen ikililer | Matris (satırlar $a, b, c, d$) |
|---|---|---|
| $k = 1$ ($a$ üzerinden) | $(b, d)$ | `0001, 1011, 1001, 0010` |
| $k = 2$ ($b$ üzerinden) | yok | `0001, 1011, 1001, 0010` |
| $k = 3$ ($c$ üzerinden) | $(d, a), (d, d)$ | `0001, 1011, 1001, 1011` |
| $k = 4$ ($d$ üzerinden) | $(a, a), (a, c), (c, c)$ | `1011, 1011, 1011, 1011` |

$k = 4$ adımından sonra matrisin bütün satırları `1011` olur. Bu matrisin grafı ilk verilen grafla hiç benzemez, çünkü geçişlilik için gereken bütün ikililer eklenmiştir. Ortaya çıkan graf geçişli bir graftır.

### 1.4 Algoritmayı Analiz Ederek Öğrenmek

Hoca, öğrenciyken geçişli kapanış ödevini yaptığını ama bu şekilde yapmadığını anlattı. Onun kodu da üç döngülüydü, ama döngülerin içinde tek bir komut satırı değil, onlarca satır vardı. Geçişliliği buluyordu ama etkili (efektif) değildi. Sonra hocaları optimize edilmiş çözümü gösterdi; sınıfta kimse bu fikri bulamamıştı.

Hocanın öğüdü: "Ben de böyle yazmalıyım" diye bir beklentiye girmeyin. Kendinizi geliştirmek istiyorsanız $3 \times 3$'lük bir matris alın, algoritmanın adımlarını üzerinde analiz edin ve bu kişinin bu formülü nasıl görebildiğini anlamaya çalışın. Bir soruyu herkes farklı bir algoritmayla çözer; temel noktalar örtüşebilir ama birebir aynı olma ihtimali yoktur.

> **Hocanın vurgusu:** Bir algoritma kitaplara girdiyse, o alanında en etkili, en optimize edilmiş algoritmadır. Sıradan algoritmalar kitaplara girmez. Her zaman kendinizle yarışın.

## 2. Matris Bağıntıları

Hoca fonksiyonlara geçip birebir ve örten kavramlarını anlatırken bir öğrenci, matris bağıntılarının işlenmediğini hatırlattı. Hoca bu konuda anlatılacak ayrı bir şey olmadığını söyledi. Matris bağıntısı, verilmiş bir bağıntıyı matris biçiminde yazmaktır. Matris $a_{ij}$ biçiminde yazılır. Satırlar tanım kümesinin, sütunlar değer kümesinin elemanlarını simgeler. $(x, y)$ bağıntıda varsa ilgili göz 1, yoksa 0 olur. Warshall örneğinde yapılan da tam olarak buydu.

## 3. Fonksiyon Kavramı

### 3.1 Tanım

Fonksiyon, $X$ kümesinden (tanım kümesi, domain) $Y$ kümesine (değer kümesi, range) bir bağıntıdır. Bu bağıntılar $X \times Y$ Kartezyen çarpımının (Cartesian product) içinden seçilir. Ama bir şart vardır:

> **Hocanın vurgusu:** Bir fonksiyonun fonksiyon olabilmesi için tanım kümesindeki her eleman, değer kümesinde **tam olarak bir** (exactly one) elemana karşılık gelmelidir. Fonksiyonu fonksiyon yapan budur, başka bir şey yoktur.

Hoca burada sık yapılan bir karışıklığa dikkat çekti. Tanım kümesindeki iki farklı eleman, değer kümesinde aynı elemana gidebilir. Örneğin $f(x) = x^2$ için $-2$ ve $2$'nin ikisi de $4$'e gider; bu serbesttir. Olmaması gereken şey, tanım kümesindeki bir elemanın değer kümesinde iki farklı

elemana gitmesidir. Çünkü $a$'yı fonksiyonda yerine koyduğumuzda "bu mu, şu mu" diye bir seçenek olamaz.

Slayttaki örnekte tanım kümesi $\{a, b, c, d\}$, değer kümesi $\{1, 3, 5\}$'tir. Oklardan $f(a) = 3$, $f(b) = 3$, $f(c) = 5$ okunur.

### 3.2 Fonksiyonlarda Toplama ve Çarpma

İki fonksiyon toplanabilir ve çarpılabilir. Slayttaki örnekte $f_1(x) = x^2$ ve $f_2(x) = x - x^2$'dir.

- **Toplam:** $(f_1 + f_2)(x) = x^2 + x - x^2 = x$. Hocanın deyişiyle $x^2$ ve $-x^2$ birbirini götürür, $x$ kalır.
- **Çarpım:** $(f_1 f_2)(x) = x^2(x - x^2) = x^3 - x^4$. Üsler dağıtılır ve yeni bir fonksiyon ortaya çıkar (ara adım eklendi).

### 3.3 Liste Biçimindeki Örnekler

- $X = \{1, 2, 3\}$, $Y = \{a, b, c\}$ ve $f = \{(1, a), (2, b), (3, a)\}$: Bu bir fonksiyondur. 1 ve 3'ün ikisi de $a$'ya gidiyor, ama bu serbesttir.
- $f = \{(1, a), (2, b), (3, c), (1, b)\}$: Bu bir fonksiyon değildir. 1, hem $a$'ya hem $b$'ye gidiyor.

## 4. Birebir, Örten ve Birebir-Örten Fonksiyonlar

Hoca, bağıntıların yansımalı, simetrik, geçişli gibi özellikleri olduğu gibi fonksiyonların da özellikleri olduğunu söyledi.

- **Birebir (one-to-one, injective):** Tanım kümesindeki her eleman değer kümesinde ayrı bir elemana karşılık gelir; iki farklı eleman aynı değere gitmez. Biçimsel olarak $f(a) = f(b) \Rightarrow a = b$.
  - $f(x) = 2^x$ (reel sayılarda) birebirdir: 1'in karşılığı 2, 2'nin karşılığı 4, 3'ün karşılığı 8'dir. Her biri ayrı bir değere gider.
  - $f(x) = x^2$ birebir değildir: $-2$ ve 2'nin ikisi de 4'ü gösterir.
- **Örten (onto, surjective):** Değer kümesinde açıkta eleman kalmaz. Değer kümesindeki her eleman en az bir elemanın görüntüsüdür.
- **Birebir-örten (bijective):** Fonksiyon hem birebir hem örtendir.

### 4.1 Ok Diyagramı Örnekleri

Hoca slayttaki ok diyagramlarını sınıfa sordu ve sırayla cevapladı. Her diyagramda önce fonksiyon olup olmadığına, sonra birebir ve örten olup olmadığına bakılır.

| Diyagram | Fonksiyon mu? | Birebir mi? | Örten mi? | Gerekçe |
|---|---|---|---|---|
| 1 | Evet | Evet | Hayır | Değer kümesinde açıkta bir eleman var. |
| 2 | Evet | Hayır | Evet | $a$ ve $d$ aynı değere gidiyor; açıkta eleman yok. |
| 3 | Evet | Evet | Evet | Birebir-örtendir. |
| 4 | Evet | Hayır | Hayır | $a$ ve $c$, 2'ye gidiyor; 4 açıkta. |
| 5 | Hayır | | | $a$ iki farklı elemana gidiyor. |

Son diyagram fonksiyon olmadığı için diğer özelliklere bakmaya gerek yoktur.

## 5. Ters Fonksiyon

Bir fonksiyonun tersi $f^{-1}$ ile gösterilir. Her fonksiyonun tersi bir fonksiyon olmayabilir.

> **Hocanın vurgusu:** Bir fonksiyon birebir-örten (bijective), yani hem örten hem birebir ise tersi de bir fonksiyondur.

### 5.1 Liste Biçimi

Fonksiyon liste (bağıntı) biçiminde verilmişse, bağıntının tersinde olduğu gibi $x$ ile $y$ yer değiştirir; tanım kümesi ile değer kümesi de yer değiştirir:

$$f = \{(1, a), (2, c), (3, b)\} \Rightarrow f^{-1} = \{(a, 1), (c, 2), (b, 3)\}$$

### 5.2 $ax + b$ Biçimi

Fonksiyon $f(x) = ax + b$ ise $a$ paydaya iner ve $b$'nin işareti değişir:

$$f(x) = ax + b \Rightarrow f^{-1}(x) = \frac{x - b}{a}$$

Hocanın örneği: $f(x) = x + 1$. $x$'in önünde 1 olduğu için payda 1'dir ve artı işareti eksiye döner: $f^{-1}(x) = x - 1$.

### 5.3 $\dfrac{ax + b}{cx + d}$ Biçimi

Bu biçimde $a$ ile $d$ yer değiştirir ve ikisinin de işareti değişir:

$$f(x) = \frac{ax + b}{cx + d} \Rightarrow f^{-1}(x) = \frac{-dx + b}{cx - a}$$

Slaytta bu biçimin örneği yoktu; hoca bir örnek yazdı:

$$f(x) = \frac{3x - 5}{x - 4}$$

Burada $a = 3$ ve $d = -4$'tür. Yer değiştirirken işaretler de değişir: $-4$ yukarıya $+4$ olarak, $+3$ aşağıya $-3$ olarak geçer.

$$f^{-1}(x) = \frac{4x - 5}{x - 3}$$

Python ile birkaç noktada $f(f^{-1}(x)) = x$ olduğu kontrol edildi.

### 5.4 Polinom Biçimi

Polinom biçiminde iş biraz daha zahmetlidir. Fonksiyon $y$'ye eşitlenir ve işlemler, $x$ eşitliğin bir tarafında yalnız kalacak şekilde düzenlenir. Örnek:

$$y = x^2 - 6x + 13$$

$x$'i yalnız bırakmanın en kolay yolu ifadeyi tam kare biçiminde yazmaktır. Çarpımları 9, toplamları $-6$ olan kökler $-3$ ve $-3$'tür. Bu yüzden 13, $9 + 4$ olarak yazılır:

1. $y = (x^2 - 6x + 9) + 4 = (x-3)^2 + 4$
2. $y - 4 = (x-3)^2$
3. $\sqrt{y-4} = x - 3$
4. $x = \sqrt{y-4} + 3$

Sonuç olarak $f^{-1}(x) = \sqrt{x-4} + 3$ olur.

> **Ders sonrası not:** $x^2 - 6x + 13$ reel sayılarda birebir değildir (örneğin $x = 2$ ve $x = 4$ aynı değeri, 5'i verir). Tersin fonksiyon olması için tanım kümesi $x \geq 3$ ile sınırlanmalıdır. O zaman $f^{-1}(x) = \sqrt{x-4} + 3$, $x \geq 4$ için tanımlıdır.

Hoca bölümü şöyle özetledi: Liste biçiminde tanım ve değer kümesi yer değiştirir. $ax + b$ biçiminde $a$ paydaya iner, ortadaki işaret değişir. Kesirli biçimde $a$ ile $d$ yer değiştirir ve işaretleri değişir. Polinom biçiminde $y$'ye eşitleyip $x$ yalnız bırakılır.

## 6. Fonksiyon Bileşkesi

### 6.1 Bağıntılardaki Bileşkeyle İlişkisi

Hoca bileşkeyi bağıntılardan hatırlattı. Bağıntılarda $R^2 = R \circ R$, $R^3 = R^2 \circ R$ idi. Bir bağıntıdan $(x, y)$, diğerinden $(y, z)$ alınır ve geçişlilikte olduğu gibi $(x, z)$ yazılırdı. Fonksiyonlarda da aynı mantık vardır: bir fonksiyon diğerinin içine yerleştirilir.

$f \circ g$ "f bileşke g" diye okunur; kısaca "fog" da denir. $(f \circ g)(x) = f(g(x))$'tir: $g$ alınır ve $f$'nin içindeki $x$'in yerine yazılır. $g \circ f$ ise "gof" diye okunur ve $(g \circ f)(x) = g(f(x))$'tir.

### 6.2 Slayttaki Örnek

$g(x) = x^2 - 1$ ve $f(x) = 3x + 5$ olsun.

$f \circ g$**:** $f$'nin içindeki $x$'in yerine $g$ yazılır.

$$(f \circ g)(x) = 3(x^2 - 1) + 5 = 3x^2 - 3 + 5 = 3x^2 + 2$$

$g \circ f$**:** $g$'nin içindeki $x$'in yerine $f$ yazılır. Kare açılırken birincinin karesi $9x^2$, birinci ile ikincinin çarpımının iki katı $30x$, ikincinin karesi $25$'tir.

$$(g \circ f)(x) = (3x + 5)^2 - 1 = 9x^2 + 30x + 25 - 1 = 9x^2 + 30x + 24$$

İki sonuç eşit değildir. Demek ki fonksiyon bileşkesinde değişme özelliği yoktur.

> **Ders sonrası not:** Hoca $f \circ g$ sonucunu sözlü olarak "$3x^2 + 1$" diye okudu. $g \circ f$ için de $(3x + 5)^2 + 1 = 9x^2 + 30x + 26$ diye hesapladı. $g(x) = x^2 - 1$ olduğu için doğru sonuçlar $3x^2 + 2$ ve $9x^2 + 30x + 24$'tür. Python ile doğrulandı. Sonuçların eşit olmadığı yargısı değişmez.

### 6.3 Birleşme Var, Değişme Yok

- **Birleşme (associativity):** $f \circ (g \circ h) = (f \circ g) \circ h$. Önce $g$ ile $h$'nin bileşkesini alıp sonra $f$ ile birleştirmek, önce $f$ ile $g$'yi birleştirip sonra $h$ ile birleştirmekle aynı sonucu verir.
- **Değişme (commutativity):** Genelde $f \circ g \neq g \circ f$'dir.

### 6.4 Üstel ve Logaritmik Fonksiyonlar

Hoca değişme özelliğinin bir durumda sağlandığını söyledi: fonksiyonlardan biri üstel, diğeri logaritmikse ve birbirinin tersiyse. Slayttaki örnek $f(x) = 2^x$ ve $g(x) = \log_2 x$'tir. İki tabanında logaritma $\lg x$ diye de yazılır.

- $(f \circ g)(x) = 2^{\lg x} = x$. Logaritma kuralına göre 2 üzeri iki tabanında logaritma birbirini götürür.

- $(g \circ f)(x) = \lg 2^x = x$. Tabanla üsteki 2 aynı olduğu için sonuç $x$'tir.

Üstel bir fonksiyonun tersi logaritmik, logaritmik bir fonksiyonun tersi üstel bir fonksiyondur. Bu yüzden bu ikisinde $f \circ g = g \circ f$ çıkar.

Arada bir öğrenci, farklı tabanlarla denediğinde eşitliğin çıkmadığını söyledi (bkz. Örnek Sorular). Hoca, üstel ve logaritmik fonksiyonların birbirinin tersi olduğunun kesin olduğunu, ama değişme özelliği konusunda aklında kalanı söylemiş olabileceğini, kitaptan kontrol edeceğini belirtti.

> **Ders sonrası not:** Eşitlik yalnız iki fonksiyon birbirinin tersiyse geçerlidir; yani üstel fonksiyonla logaritmanın tabanı aynı olmalıdır. Örneğin $2^x$ ile $\log_3 x$ birbirinin tersi değildir ve $f \circ g \neq g \circ f$ olur. Ayrıca $2^{\lg x} = x$ eşitliği yalnız $x > 0$ için tanımlıdır.

## 7. Dizginin Tersi

Bir dizgi (string) verildiğinde tersini almak kolaydır: tersten yazılır. $\alpha = a_1a_2a_3a_4$ ise $\alpha$'nın tersi $a_4a_3a_2a_1$'dir. Bir fonksiyonun tersinin tersi kendisine eşit olduğu gibi, bir dizgiyi iki kez ters çevirmek de dizginin kendisini verir.

Hoca burada $2\alpha$ ile $\alpha^{-1}$'i 2 ile $\frac{1}{2}$'ye benzetip "birbirini götürür, sonuç $\lambda$'dır" dedi; sonra bu örneği geri alıp "bir dizginin kendisiyle tersinin çarpımı boş kümedir" dedi.

> **Ders sonrası not:** $\lambda$ boş küme değil, **boş dizgidir** (uzunluğu 0 olan dizgi). Ayrıca dizginin ters çevrilmişi (reversal), dizgiyle art arda yazılınca (birleştirme) boş dizgiyi veren bir "ters eleman" değildir. $a_1a_2$ ile $a_2a_1$ birleştirilince $a_1a_2a_2a_1$ çıkar, boş dizgi çıkmaz.

## 8. Taban (Floor) ve Tavan (Ceiling) Fonksiyonları

Hoca bu iki fonksiyonun çok önemli olduğunu söyledi. "Floor" yer döşemesidir; Türkçede **taban** denir. "Ceiling" ise **tavan**dır.

Hoca bu gösterimi ilk gördüğünde, köşeli parantezin yalnız alt ya da üst çizgisi olduğu için kitapta hata olduğunu sandığını anlattı. Sonra konuyu öğrenince hiç alakası olmadığını gördüğünü söyledi. Öğüdü: Bilmediğimiz bir gösterime hemen "yanlış" damgası vurmamalıyız.

### 8.1 Taban

$\lfloor x \rfloor$: $x$'e eşit veya ondan küçük en büyük tam sayıdır.

Hoca bunu sayı ekseniyle anlattı. Eksende 3, 4 ve 5 işaretlensin; 4.2 sayısı 4 ile 5 arasındadır. 4.2'ye eşit veya ondan küçük tam sayılar 4, 3, 2, … dir; bunların en büyüğü 4'tür. Yani $\lfloor 4.2 \rfloor = 4$.

Sayı 4.9 olsa bile, 5'e çok yakın olmasına rağmen sonuç değişmez: $\lfloor 4.9 \rfloor = 4$.

### 8.2 Taban Yuvarlama ya da Kesme Değildir

> **Hocanın vurgusu:** Tabanı yuvarlama (round) ya da kesme (trunc) ile karıştırmayın.

- **Yuvarlama (round):** Programlama dillerinde görülür. Noktadan sonrası 5 veya 5'ten büyükse yukarı, küçükse aşağı yuvarlanır. 4.5 ve 4.6 için sonuç 5, 4.2 için 4'tür.
- **Kesme (trunc):** Noktadan sonrası kesilip atılır. 4.2, 4.5 ve 4.6'nın hepsi 4 olur.
- **Taban:** Yalnız tanıma bakılır: eşit veya küçük en büyük tam sayı.

  **Ek bilgi (derste anlatılmadı):** Pozitif sayılarda kesme ile taban aynı sonucu verir, negatif sayılarda vermez. Python'da `math.trunc(-8.7)` sonucu $-8$, `math.floor(-8.7)` sonucu $-9$'dur. Ayrıca yuvarlama kuralı dilden dile değişebilir: Python'un `round` fonksiyonu tam yarıda kalan sayıyı çift sayıya yuvarlar, bu yüzden `round(4.5)` sonucu 4'tür.

### 8.3 Tavan

$\lceil x \rceil$: $x$'e eşit veya ondan büyük en küçük tam sayıdır. Taban fonksiyonunun tam tersidir.

Hoca eksende 2, 3 ve 4'ü işaretledi ve 3.5'i yerleştirdi. 3.5'e eşit veya ondan büyük tam sayıların en küçüğü 4'tür: $\lceil 3.5 \rceil = 4$. Sayı 3.1 ya da 3.9 olsa da sonuç yine 4 olurdu.

### 8.4 Slayttaki Alıştırmalar

Hoca slayttaki örnekleri sınıfa sordu, cevaplar birlikte bulundu. Hepsi Python ile doğrulandı.

| İfade | Sonuç | Açıklama |
|---|---|---|
| $\lfloor 8.3 \rfloor$ | 8 | 8.3'ten küçük en büyük tam sayı. |
| $\lfloor 1/2 \rfloor$ | 0 | 0.5, 0 ile 1 arasında. |
| $\lfloor -8.7 \rfloor$ | $-9$ | $-8.7$, $-9$ ile $-8$ arasında; $-8$ ondan büyüktür. |
| $\lfloor 7 \rfloor$ | 7 | Tanımdaki "eşit" durumu. |
| $\lfloor -1/2 \rfloor$ | $-1$ | $-0.5$'ten küçük en büyük tam sayı. |
| $\lfloor 3.1 \rfloor$ | 3 | |
| $\lceil -11.3 \rceil$ | $-11$ | $-11.3$'ten büyük en küçük tam sayı. |
| $\lceil 6 \rceil$ | 6 | Eşit durumu. |
| $\lceil -8 \rceil$ | $-8$ | Eşit durumu. |
| $\lceil 1/2 \rceil$ | 1 | |
| $\lceil -1/2 \rceil$ | 0 | $-0.5$'ten büyük en küçük tam sayı. |
| $\lceil 3.1 \rceil$ | 4 | |

Negatif sayılarda dikkat edilmelidir: $\lfloor -8.7 \rfloor = -9$'dur, çünkü $-9 < -8.7$'dir.

Alıştırmaların sonunda bir öğrenci devirli sayının tabanını sordu: $\lfloor 1.\overline{9} \rfloor$ nedir? Soru ve cevabı Örnek Sorular bölümündedir.

## 9. Bölüm Soruları: Küme İşlemleri ve Venn Şeması

Arada sonra hoca, kümeler, fonksiyonlar ve bağıntılar bölümünün soru-cevap slaytlarına geçti. Amaç bir buçuk iki haftadır anlatılanları hızlıca hatırlamaktı.

### 9.1 Küme İşlemleri

Evrensel küme $U = \{1, 2, \ldots, 10\}$, $A = \{1, 4, 7, 10\}$, $B = \{1, 2, 3, 4, 5\}$, $C = \{2, 4, 6, 8\}$'dir.

- $A \cup B$**:** İki kümenin bütün elemanları bir araya getirilir, ortak elemanlar (1 ve 4) bir kez yazılır: $\{1, 2, 3, 4, 5, 7, 10\}$.
- $B - A$**:** $B$'nin içinden $A$'nın elemanları çıkarılır. 1 ve 4 çıkar; 7 ve 10 zaten $B$'de yoktur: $\{2, 3, 5\}$.
- $\overline{U}$**:** Evrensel kümenin tümleyeni, yani bütün dünya 1 ile 10 arasındaki sayılarsa bunun dışındaki her şey: $\emptyset$.
- $A \cup U$**:** $A$ ne olursa olsun, evrensel kümeyle birleşimi evrensel kümedir: $U$.
- $\overline{B} \cap (C - A)$**:** Önce parantezin içi. $C$'den $A$'nın elemanı olan 4 çıkar: $C - A = \{2, 6, 8\}$. $\overline{B} = \{6, 7, 8, 9, 10\}$. Kesişim $\{6, 8\}$'dir.
- $(A \cup B) - (C - B)$**:** $A \cup B = \{1, 2, 3, 4, 5, 7, 10\}$. $C$'den $B$'nin elemanları 2 ve 4 çıkar: $C - B = \{6, 8\}$. $A \cup B$'nin içinde 6 ve 8 olmadığı için sonuç $A \cup B$'nin kendisidir: $\{1, 2, 3, 4, 5, 7, 10\}$.

### 9.2 Venn Şemasıyla Gölgelendirme

Slaytta önce çözülmüş bir örnek vardı: $A$ kümesi ile $\overline{B}$'nin (B'nin dışında kalan yerlerin) kesişimi gölgelendirilmişti.

Ardından daha karmaşık bir ifade soruldu:

$$\Big((C \cap A) - \overline{(B - A)}\Big) \cap C$$

Hoca önce iç parantezleri gölgelendirmeyi, sonra adım adım dışa doğru gitmeyi önerdi. Çizim sırasında bir karışıklık oldu; hoca bölgeleri tekrar çizip şöyle çözdü:

1. $C \cap A$, $C$ ile $A$'nın ortak bölgesidir. Bu bölge $A$'nın içindedir.
2. $B - A$, $B$'nin $A$ dışında kalan kısmıdır. Tümleyeni $\overline{(B - A)}$ bunun dışındaki her yerdir; $A$'nın tamamı da buna dahildir.
3. $C \cap A$ bölgesinden $\overline{(B - A)}$ çıkarılınca hiçbir şey kalmaz, çünkü $C \cap A$'nın tamamı $\overline{(B - A)}$'nın içindedir.
4. Boş kümenin $C$ ile kesişimi de boştur.

Sonuç $\emptyset$'dir. Python ile yukarıdaki kümeler üzerinde de kontrol edildi.

## 10. Bölüm Soruları: Üç Kümeli Sayma Problemi

Hoca, slayt değişeceği için verilerin not edilmesini istedi:

- Toplam 191 öğrenci var.
- 65 kişi Fransızca (F), 76 kişi işletme (business, B), 63 kişi müzik (M) dersi alıyor.
- 36 kişi F ve B, 20 kişi F ve M, 18 kişi B ve M alıyor.
- 10 kişi üç dersi birden alıyor.

Soru: Kaç kişi Fransızca ve müzik alıp işletme almıyor?

Bu tip sorular Venn şemasıyla ya da formülle çözülebilir. Hoca Venn şemasını kullandı. Bilgiler **en içteki bölgeden başlayarak** yerleştirilir:

1. Üçünü birden alanlar: 10.
2. F ve B alan 36 kişinin 10'u müzik de alıyor. Yalnız F ve B alan: $36 - 10 = 26$.
3. F ve M alan 20 kişinin 10'u işletme de alıyor. Yalnız F ve M alan: $20 - 10 = 10$.
4. B ve M alan 18 kişinin 10'u Fransızca da alıyor. Yalnız B ve M alan: $18 - 10 = 8$.
5. Yalnız F: $65 - (26 + 10 + 10) = 65 - 46 = 19$.
6. Yalnız B: $76 - (26 + 10 + 8) = 76 - 44 = 32$.
7. Yalnız M: $63 - (10 + 10 + 8) = 63 - 28 = 35$.

Sorunun cevabı, F ve M bölgesinin işletme almayan kısmıdır: **10**.

> **Hocanın vurgusu:** Venn şeması bu şekilde bütün bölgeleriyle doldurulursa hangi soru gelirse gelsin cevap şemadan okunur. Formülü kullansanız da aynı cevabı bulursunuz.

Hoca örnek olarak başka sorular da türetti. Yalnız bir ders alanların sayısı $19+32+35 = 86$'dır. Şemadaki bölgelerin toplamı $19 + 32 + 35 + 26 + 10 + 8 + 10 = 140$'tır; dolayısıyla hiç ders almayan $191 - 140 = 51$ kişidir (bu son soru derste sorulmadı, şemadan hesaplandı). Hoca ayrıca, çözümleri PDF'te olan öğrencilerin önce kendilerinin çözmesini, yalnız yapamadıkları sorularda cevaplara bakmasını önerdi; böylece nereyi anlamadıkları ortaya çıkar.

> **Ders sonrası not:** Hoca bölgeleri yerleştirirken iki kez sayı karıştırdı: Fransızca toplamını "76" diye söyledi (doğrusu 65) ve yalnız F ve M bölgesi için "kalan yirmi kişi" dedi (doğrusu 10). Yukarıdaki değerler verilerle tutarlıdır ve Python ile doğrulandı.

## 11. Bölüm Soruları: Doğru/Yanlış ve Karşı Örnek

Soru: $U$ evrensel küme, $X$ ve $Y$ onun alt kümeleri olsun. Her $X$ ve $Y$ için

$$(X \cap Y) \cup (Y - X) = X$$

doğru mudur? Yanlışsa karşı örnek (counterexample) verin. Karşı örnek, bir ifadenin yanlış olduğunu gösteren tek bir örnektir.

Hoca önce şemayı çizip ifadenin doğru olduğunu söyledi. Sonra slayttaki cevabın "yanlış" olduğunu görünce çizimi baştan yaptı:

1. $Y - X$: $Y$'nin $X$ dışında kalan kısmı.
2. $X \cap Y$: $X$ ile $Y$'nin ortak kısmı.
3. Bu iki bölgenin birleşimi $Y$'nin tamamıdır.

Yani sol taraf her zaman $Y$'ye eşittir, $X$'e değil. İfade **yanlıştır**. Slaytta bir karşı örnek de vardı, ama altyazıda okunmuyor (çözüm için bkz. Örnek Sorular).

## 12. Bölüm Soruları: Dizi Toplamları ve Sayı Tabanları

### 12.1 İlk Dizi

Dizi $a_i = i^2 - 3i + 3$, $i \geq 1$ olarak verilmiştir. İstenen $\sum_{i=1}^{4} a_i$'dir.

| $i$ | $i^2$ | $-3i$ | $+3$ | $a_i$ |
|---|---|---|---|---|
| 1 | 1 | $-3$ | 3 | 1 |
| 2 | 4 | $-6$ | 3 | 1 |
| 3 | 9 | $-9$ | 3 | 3 |
| 4 | 16 | $-12$ | 3 | 7 |

Toplam $1 + 1 + 3 + 7 = 12$'dir. Hoca ilk hesapta 13 buldu; öğrencilerin uyarısıyla $a_4$'ü düzeltti ve 12'ye ulaştı.

Aynı dizi için ikinci bir soru da vardı. Hoca $a_3 = 3$ ile $a_4 = 7$'yi çarparak 21'i buldu: $3 \cdot 7 = 21$.

### 12.2 İç İçe Toplam

$y_i = 2^i - 1$ ve $z_i = i(i-1)$ olsun. İfade, $i = 1$'den 3'e kadar $y_i$'lerin, içteki $z_i$ toplamıyla çarpılıp toplanmasıdır. Hoca önce içteki toplamı hesapladı:

$$\sum_{i=1}^{3} z_i = 1 \cdot 0 + 2 \cdot 1 + 3 \cdot 2 = 0 + 2 + 6 = 8$$

Sonra dıştaki toplamı:

$$\sum_{i=1}^{3} 8\, y_i = 8 \cdot 1 + 8 \cdot 3 + 8 \cdot 7 = 8 + 24 + 56 = 88$$

Bu, $(\sum y_i)\,(\sum z_i) = 11 \cdot 8 = 88$ ile aynıdır. Slayttaki cevap da 88'dir. Python ile doğrulandı. Aynı slayttaki son şıkkı hoca öğrencilere bıraktı.

### 12.3 Sayı Tabanları

> **Hocanın vurgusu:** Bu soruda çok dikkatli olun.

**9450 hangi tabanda bir sayı olabilir?** Bir sayının bir tabanda olabilmesi için rakamlarının tabandan küçük olması gerekir.

- İkili (binary) olamaz: İkili sayılar yalnız 0 ve 1'den oluşur.
- Sekizli (octal) olamaz: Rakamlar 8'den küçük olmalı, ama 9 var.

- Onlu (decimal) olabilir: Rakamlar 0 ile 9 arasında.
- On altılı (hexadecimal) olabilir: 0'dan 15'e kadar değerler kullanılır; 10'dan sonrası harflerle yazılır.

Günlük hayatta bu sayı onlu kabul edilir. Onludan farklı bir tabandan söz ediyorsak tabanı mutlaka yazmamız gerekir.

**1101010 onlu mu, on altılı mı?** Bu sayı onlu olarak okunabilir: "1.101.010". Bütün rakamları 16'dan küçük olduğu için on altılı da olabilir. Yine de işin raconu, 10'dan farklı bir taban kastediliyorsa tabanın yazılmasıdır.

> **Ders sonrası not:** Hoca on altılı rakamları sayarken "D 14" dedi. Doğrusu $A = 10$, $B = 11$, $C = 12$, $D = 13$, $E = 14$, $F = 15$'tir. Ayrıca 1101010 yalnız 0 ve 1'den oluştuğu için ikili bir sayı da olabilir.

## 13. Bölüm Soruları: Bağıntılar ve Ters Fonksiyon

### 13.1 $x + y \leq 6$ Bağıntısı

$\{1, 2, 3, 4, 5\}$ üzerinde $R = \{(x, y) \mid x + y \leq 6\}$ bağıntısının elemanları yazılacak ve özellikleri incelenecek.

Hoca önce yalnız toplamı 6'ya eşit olan ikilileri yazmaya başladı, sonra "küçüktür" şartını da hatırlayıp düzeltti. Doğru liste:

- 1 ile: $(1,1), (1,2), (1,3), (1,4), (1,5)$ (toplamlar 2, 3, 4, 5, 6)
- 2 ile: $(2,1), (2,2), (2,3), (2,4)$
- 3 ile: $(3,1), (3,2), (3,3)$
- 4 ile: $(4,1), (4,2)$
- 5 ile: $(5,1)$

Toplam 15 ikili vardır. Özellikler:

- **Yansımalı değil:** $(1,1), (2,2), (3,3)$ var, ama $(4,4)$ ve $(5,5)$ yok.
- **İrrefleksif değil:** İrrefleksif olması için hiçbir $(x, x)$ olmamalıydı; bazıları var. Hoca bu durumu "non-reflexive" diye adlandırdı: bazı $(x, x)$'ler var, bazıları yok.
- **Simetrik:** $(5,1)$ varken $(1,5)$, $(4,2)$ varken $(2,4)$ da var. Toplamada değişme özelliği olduğu için $(x, y)$ şartı sağlıyorsa $(y, x)$ de sağlar.
- **Ters simetrik değil:** $(3,2)$ ve $(2,3)$ ikisi birden var.
- **Geçişli değil:** $(5,1)$ ve $(1,2)$ var; geçişlilik için $(5,2)$ de olmalıydı. Ama $5 + 2 = 7 > 6$ olduğu için $(5,2)$ bağıntıda olamaz.

Bütün özellikler Python ile kontrol edildi.

> **Ders sonrası not:** Hoca yansımalılığı anlatırken "$(6,6)$ yok" da dedi. 6 kümenin elemanı değildir; eksik olan yalnız $(4,4)$ ve $(5,5)$'tir.

### 13.2 Ters Simetrik Bağıntıların Birleşimi

Soru: "$R$ ve $S$ ters simetrik (antisymmetric) ise $R \cup S$ de ters simetriktir." Doğru mu? Yanlışsa karşı örnek verin.

Hocanın çözümü: Öyle iki bağıntı yazılmalı ki ikisi ayrı ayrı ters simetrik olsun, ama birleşimi olmasın. $R = \{(1,2)\}$ ve $S = \{(2,1)\}$ alınır. $R$'de $(1,2)$ var, $(2,1)$ yok; ters simetriktir. $S$'de $(2,1)$ var, $(1,2)$ yok; o da ters simetriktir. Birleşim $\{(1,2), (2,1)\}$'dir ve simetriktir, ters simetrik değildir. İfade **yanlıştır**.

### 13.3 Denklik Bağıntısı Soruları

Hoca tanımları hatırlattı: Yansımalı, simetrik ve geçişli olan bağıntı **denklik bağıntısıdır** (equivalence relation).

> **Ders sonrası not:** Hoca sıralama bağıntısını "simetrik, ters simetrik ve geçişli" diye tanımladı. Doğrusu: **yansımalı**, ters simetrik ve geçişli olan bağıntı (kısmi) sıralama bağıntısıdır.

Slayttaki ilk bağıntının üç özelliği de sağladığı ve denklik bağıntısı olduğu söylendi; ayrıntıyı hoca öğrencilere bıraktı.

İkinci soru: $\{1, 2, 3, 4, 5\}$ üzerinde "3, $x+y$'yi böler" (3 divides $x+y$) bağıntısı denklik bağıntısı mıdır? Öyle $(x, y)$ çiftleri yazılacak ki $x + y$, 3'e tam bölünsün. Hoca şu çiftleri yazdı:

- $1 + 1 = 2$ bölünmez; $(1, 2)$: 3 bölünür; $(1, 5)$: 6 bölünür.
- $(2, 4)$: 6 bölünür.
- Terslerini de alarak: $(4, 2)$, $(5, 1)$, $(2, 1)$.

$(1, 1)$ bağıntıda olmadığı için yansımalı özelliği yoktur. Yansımalı olmayan bir bağıntı ne denklik ne de sıralama bağıntısı olabilir. Cevap: denklik bağıntısı **değildir**.

> **Ders sonrası not:** Hoca 3 için "4 ve 5 ile toplanınca 7 ve 8 olur, bölünmez" dedi ve $(3, 3)$'ü atladı; $3+3 = 6$ olduğu için $(3, 3)$ bağıntıdadır. $(4, 5)$ ve $(5, 4)$ da $(4+5 = 9)$ eksik kaldı. Doğru bağıntı 9 çifttir: $(1, 2), (1, 5), (2, 1), (2, 4), (3, 3), (4, 2), (4, 5), (5, 1), (5, 4)$. $(1, 1)$ olmadığı için sonuç değişmez.

### 13.4 Ters Fonksiyon Soruları

Slayttaki fonksiyonların her biri birebirdir ve tersleri istenmektedir.

- $f(x) = 4x + 2$: Kurala göre $x$'in önündeki 4 paydaya gelir, 2'nin işareti değişir: $f^{-1}(x) = \dfrac{x - 2}{4}$.
- $f(x) = 3 + \dfrac{1}{x}$: Hoca cevabı sınıftan bekledi. Bir öğrencinin önerdiği cevap doğru çıkmadı. Slayttaki cevap $f^{-1}(x) = \dfrac{1}{x - 3}$'tür.

  **Ders sonrası not:** Hoca açıklarken fonksiyonu bir an "$0.5x + 3$" gibi bir ifadeyle anlattı ve konu karıştı. Doğru çözüm: $y = 3 + \dfrac{1}{x} \Rightarrow y - 3 = \dfrac{1}{x} \Rightarrow x = \dfrac{1}{y - 3}$. Yani $f^{-1}(x) = \dfrac{1}{x - 3}$ (ara adım eklendi). Python ile $f(f^{-1}(x)) = x$ olduğu kontrol edildi.

## 14. Algoritmalar Bölümüne Giriş

### 14.1 Bölümün Kapsamı

Hoca, algoritmalar bölümünün bu dönem derse yeni eklendiğini, yüz küsur slayt olduğunu ve vizeye kadar hepsinin yetişip yetişmeyeceğini bilmediğini söyledi. Bölümün eklenmesinin bir sebebi, öğrencilerin birinci sınıfın ikinci dönemini çevrimiçi geçirmiş olmasıydı; algoritmalar konusunu bu dersle desteklemek istemişlerdi.

Bölümde işlenecek konular:

- Algoritma kavramı
- Temel veri yapıları ve problemler: ağaçlar (tree), yığıtlar (stack), aritmetik işlemlerin farklı gösterimleri
- Fonksiyonların büyümesi ve asimptotik gösterim (asymptotic notation)
- Algoritmanın karmaşıklığı ve yürütme zamanı (running time) nasıl hesaplanır
- Polinom zamanda çözülebilen ve çözülemeyen problemler
- Yinelemeli (iterative) ve özyinelemeli (recursive) algoritmaların analizi
- Rekürans bağıntıları (recurrence relations)

### 14.2 El-Harezmi

Algoritma kavramını ilk ortaya atan El-Harezmi'dir. "Algoritma" sözcüğü onun adından gelir. Kavram ortaya atılmış, ama hayatımıza girmesi 20. yüzyılı bulmuştur.

> **Ders sonrası not:** Hoca El-Harezmi'yi "Türk kökenli bir matematikçi" diye tanıttı. El-Harezmi, adını aldığı Harezm bölgesindendir (bugünkü Özbekistan-Türkmenistan

çevresi). Kaynakların çoğu onu büyük olasılıkla İranlı (Fars) bir bilgin olarak tanıtır; kökeni tartışmalıdır.

### 14.3 Tanım

> **Hocanın vurgusu:** Algoritma, bir hesaplamayı yapmak ya da bir problemi çözmek için sonlu ve kesin komutlar kümesidir. Algoritmayı en iyi açıklayan tanım budur.

Algoritma adım adım (step by step) çalıştırılan bir komutlar kümesidir. Sonlu sayıda bir girişi (input) vardır ve bu girişlere karşılık mutlaka bir çıkış (output) üretmelidir.

## 15. En Büyük Elemanı Bulma ve Sözde Kod

### 15.1 Sözlü Anlatım

En basit örneklerden biri, bir tam sayı dizisinin en büyük elemanını bulmaktır. Dizi: $3, 5, 10, 1, 15, 7, 2$.

Sokaktaki bir insana sorsanız dizine bakar ve "en büyük sayı 15" der. Bu doğrudur. Hocaya göre biz de aslında gözümüzle aynı şeyi yaparız, ama bilgisayarın bunu bulabilmesi için adım adım karşılaştırma yapması gerekir:

1. Dizinin ilk elemanı geçici en büyük seçilir: max $= 3$.
2. Sıradaki eleman geçici en büyükle karşılaştırılır; daha büyükse yeni geçici en büyük olur.
   - $5 > 3$, max $= 5$.
   - $10 > 5$, max $= 10$.
   - $1 < 10$, değişmez.
   - $15 > 10$, max $= 15$.
   - $7 < 15$ ve $2 < 15$, değişmez.
3. Dizi bitince giriş biter. max'ta ne kaldıysa çıkış odur: 15.

### 15.2 Bir Algoritma Nasıl Belirtilir?

Algoritma yazarken kimi akış diyagramı çizer, kimi bildiği bir dille yazar. Programlama dilinin adını taşıyan kitaplar (C ile algoritma, Python ile algoritma gibi) örnekleri o dilde verir. Genel bir algoritma kitabı ise dile bağlı anlatmaz; **sözde kod** (pseudocode) kullanır.

Sözde kod doğal dile benzer ama doğal dil değildir; programlama dillerine benzer ama programlama dili değildir. Algoritma adımlarını Türkçe yazarsanız, bir başkası İngilizce ya da Fransızca yazabilir. Sözde kod ise daha evrenseldir. Algoritmayı ve herhangi bir programlama dilini bilen biri onu rahatça anlar, sokaktaki bir insan anlamaz.

Hoca bunu bir dikiş kalıbına (patron) benzetti. Aynı sözde kodu on kişiye verip birine C, birine Java, birine Python ile yazmasını söyleyebilirsiniz. Aynı patronu birine kadife, birine ipek, birine basma, birine yünlü kumaşla verirsiniz; dikişin zorluğu değişebilir, ama model aynı modeldir. Sözde koda bakarak akış diyagramı da rahatça çizilebilir.

> **Hocanın vurgusu:** Akış diyagramıyla algoritma öğrenmek bölümün bir ekolüdür. Birçok üniversitede öğrencilerin akış diyagramı çizmeden hemen kodlamaya geçtiğini görüyoruz; algoritmaya yeni başlarken bunun doğru olmadığını savunuyoruz. Belli bir seviyeye geldikten sonra akış diyagramı çizmeyebilirsiniz, ama kod yazmadan önce kabaca sözde kodu yazın. Takıldığınız bir yeri akış diyagramına açıp analiz ederek uç nokta problemlerini yakalayabilirsiniz.

### 15.3 Sözde Kod

```
procedure max(a1, a2, ..., an: integers)
max := a1
for i := 2 to n
    if max < ai then max := ai
return max
```

Hoca sözde kodu satır satır okudu. Bu kod olduğu gibi bir derleyicide çalıştırılırsa sözdizimi (syntax) hatası verir, ama bakan herkes şunları görür:

- Bir prosedür yazılmış. Prosedüre verilen ad yaptığı işle özdeş olmalıdır: burada `max`.
- Parametre, $n$ elemanlı bir tam sayı dizisidir.
- Dizinin ilk elemanı `max` değişkenine atanır.
- Döngü 2'den $n$'e kadar döner. Her adımda $a_i$, `max`'tan büyükse `max`'ın yeni değeri $a_i$ olur; değilse `if`'e girilmez.
- Döngü bitince `return max` ile en büyük değer dışarıya döndürülür.

Dizinin ilk elemanı zaten en büyükse, `max`'ta durur ve döngü hiç `if`'in içine girmez; sonuç yine doğrudur. Bu algoritmanın girişi var, sonlu bir çıkışı var ve uç durum sorunu yoktur.

## 16. Bir Algoritmanın Özellikleri

Slayttaki özellikler ve hocanın açıklamaları:

- **Giriş (input):** Her algoritmanın belirli bir kümeden, sonlu sayıda giriş değeri vardır.
- **Çıkış (output):** Her algoritma bir çıkış üretir. Bu çıkış algoritmadan istenen çözümdür.
- **Doğruluk (correctness):** Her girişe karşı doğru çıkış üretilmelidir. "Bir girişte çalışıyor, diğerinde çalışmıyor" durumu olamaz.
- **Sonluluk (finiteness):** Her giriş için sınırlı sayıda adımdan sonra çıkış üretilmelidir.
- **Etkililik (effectiveness):** Algoritmanın her adımı doğru olarak ve sınırlı bir süre içinde yapılabilmelidir.
- **Genellik (generality):** Algoritma, istenen formdaki bütün problemler için çalışmalıdır.

Hoca genellik konusunda uç nokta (edge case) problemlerine dikkat çekti. Bazen yazdığımız kodun doğru çalıştığını düşünürüz; verdiğimiz bütün girişlerde doğru sonuç üretir. Ama bazı uç durumlar düşünülmeden kodlanmışsa, o durum geldiğinde algoritma başarısız olur (fail).

> **Hocanın vurgusu:** Uç nokta problemleri belki hayatınız boyunca karşınıza hiç çıkmayabilir, ama programı yazarken onları mutlaka düşünmek zorundasınız.
>
> **Ders sonrası not:** Hoca sonluluğu "sonsuz tane giriş hiçbir zaman verilemez" diye açıkladı. Slayttaki tanım doğrudur: Mesele girişin değil, algoritmanın **adım sayısının** sonlu olmasıdır. Hoca etkililiği ise "aynı işi biri 10 saniyede, biri 10 dakikada yapıyorsa bu kabul edilemez" diye anlattı. Bu verimlilik (efficiency) kavramıdır. Etkililik (effectiveness), her adımın kesin olarak ve sonlu sürede yapılabilmesidir.

## 17. Problem Türleri ve Doğrusal Arama

### 17.1 Üç Problem Türü

Hoca problemleri üç başlık altında topladı:

- **Arama (search)**
- **Sıralama (sort)**
- **Optimizasyon (en iyileme):** Endüstri mühendisliği için çok önemli bir problemdir. Amaç bir şeyin en büyük ya da en küçük noktasını bulmaktır; örneğin en yüksek verimi en az parayla almak.

Bütün arama ve sıralama algoritmaları burada anlatılmayacak; bunlar veri yapıları ve algoritma analizi derslerinde çok daha ayrıntılı görülecek.

### 17.2 Arama Nedir?

Elimizde bir veri yapısı vardır. Bu bir dizi olmak zorunda değildir; bir matris ya da başka bir yapı da olabilir. Amaç, bu yapının içinde bir $x$ elemanının olup olmadığını bulmaktır. Varsa hangi gözde olduğu söylenir, yoksa "yok" denir. Derste dizi üzerinde çalışıldı.

Arama algoritmaları ikiye ayrılır: doğrusal arama (linear search) ve ikili arama (binary search).

Doğrusal arama hiç optimize edilmemiş bir aramadır. Bin elemanlı bir dizide sırayla bakılır: aranan sayı birinci eleman mı, ikinci mi, üçüncü mü? Çok şanssızsanız sayıyı $n$'inci elemanda bulursunuz.

### 17.3 Sözde Kod

```
procedure linear search(x: integer, a1, a2, ..., an: distinct integers)
i := 1
while (i <= n and x != ai)
    i := i + 1
if i <= n then location := i
else location := 0
return location
```

Dizi birbirinden farklı (distinct) tam sayılardan oluşur. Hoca `while` döngüsünün nasıl çalıştığını hatırlattı: Şart doğru olduğu sürece döngünün içindeki işlem yapılır; şart yanlış olunca döngüden çıkılır. `while` içindeki sayaç mutlaka döngüde değiştirilmelidir. Problemin gerektirdiğine göre sayaç artırılır ya da azaltılır.

Bu döngüde iki şart vardır: $i \leq n$ (dizinin sonuna gelinmedi) ve $x \neq a_i$ (sayı henüz bulunmadı). Sayı bulunduğunda ikinci şart yanlış olur ve döngüden çıkılır. Döngüden çıkıldığında $i \leq n$ ise sayı $i$'nci gözdedir; değilse `location` 0 olur, yani sayı dizide yoktur.

### 17.4 Örnek

Dizi: $1, 3, 5, 6, 10, 2, 7, 16, 4, 15$ ($n = 10$; sıralı değildir). Aranan sayı: 14.

1. $i = 1$: $a_1 = 1 \neq 14$ ve $1 \leq 10$. Döngüye girilir, $i = 2$.
2. $i = 2$: $a_2 = 3 \neq 14$. $i = 3$.
3. $i = 3$: $a_3 = 5 \neq 14$. $i = 4$.
4. Bu şekilde devam edilir. 14 dizide olmadığı için hiçbir adımda bulunmaz.
5. $i = 11$ olunca $i \leq n$ şartı yanlış olur ve döngüden çıkılır.
6. $11 \leq 10$ olmadığı için `else` kısmına düşülür: `location := 0`. Sayı bulunamadı.

> **Ders sonrası not:** İzleme sırasında hoca "$i$ 15 oldu, 15, 14'ten küçük eşit mi" diye bir karışıklık yaptı. Döngüde $i$, aranan sayı $x$ ile değil dizinin eleman sayısı $n$ ile karşılaştırılır. Bu örnekte döngüden $i = 11$ iken çıkılır, çünkü $11 > n = 10$'dur.

### 17.5 Neden `for` Değil de `while`?

Hoca sınıfa sordu: Burada `while` yerine ne kullanılabilirdi? Cevap `for`'dur. 1'den $n$'e kadar dönen bir `for` döngüsü ve içinde $a_i$'nin $x$'e eşit olup olmadığına bakan bir `if` yazılabilir. İki çözümde de iki şart vardır: `while`'daki `and`'in iki parçası, ya da `for`'un sınırı ve `if`'in şartı.

Fark şudur: `for` ile sayıyı belki üçüncü elemanda bulursunuz, ama döngü yine $n$'e kadar gereksiz yere döner. Bundan kaçmak için `exit` ya da `goto` kullanmak gerekir.

> **Hocanın vurgusu:** `exit` ve `goto` kullanmak racona terstir. İyi algoritma yazan kişi bunları kullanmaz. Bulduğunuzda ya da sınıra geldiğinizde döngünün kendiliğinden bitmesi için `while` kullanın.

`for` ile yazmak yanlış sonuç vermez, ama algoritma daha uzun sürede çalışır, yani etkili olmaz.

## 18. İkili Arama

### 18.1 Ön Şart: Sıralı Dizi

> **Hocanın vurgusu:** İkili aramayı doğrusal aramadan ayıran şey yalnız aralığı yarıya bölmek değildir. İkili aramanın çalışabilmesi için dizinin mutlaka **sıralı** olması gerekir.

Dizi sıralı değilse önce sıralamak gerekir. "İki işlem yapacağıma doğrudan doğrusal arama yaparım" demek mantıklı görünebilir. Elinizde 50 elemanlı bir dizi varsa doğrusal arama ile ikili arama arasındaki fark milisaniyeler düzeyindedir. Ama bin ya da on bin elemanlı bir dizide önce sıralayıp sonra ikili arama yapmak daha etkilidir.

Yüzlerce sıralama algoritması vardır ve hepsinin karmaşıklığı ve çalışma süresi farklıdır. Karmaşıklığı düşük, hızlı bir sıralama seçilmelidir.

**Ders sonrası not:** Hoca bazı sıralamaların karmaşıklığının "logaritmik, $\log n$" olduğunu söyledi. Karşılaştırmaya dayalı sıralamalar en iyi durumda $O(n \log n)$ zamanda çalışır; hoca büyük olasılıkla $n \log n$'i kastetti. Bir diziyi $\log n$ zamanda sıralamak mümkün değildir, çünkü her elemana en az bir kez bakmak gerekir.

### 18.2 Fikir

Sıralı dizi: $1, 2, 3, 4, 5, 6, 7, 8, 13$. Doğrusal aramada 13 aranırsa "1, 13'e eşit mi? 2 eşit mi? ..." diye sona kadar gidilir; $n$ kez sorulur.

İkili arama diziyi tam ortadan ikiye böler. Aranan sayı ortadaki elemandan küçük ya da eşitse sol yarıda, büyükse sağ yarıda aramaya devam edilir. Yeni aralık yine ikiye bölünür. Böylece her adımda dizinin yarısı hiç kullanılmadan elenir.

### 18.3 Sözde Kod

```
procedure binary search(x: integer, a1, a2, ..., an: increasing integers)
i := 1        { aralığın sol ucu }
j := n        { aralığın sağ ucu }
while i < j
    m := floor((i + j) / 2)
    if x > am then i := m + 1
    else j := m
if x = ai then location := i
else location := 0
return location
```

Dizi artan sırada verilmiştir. $i$ ve $j$, aralığın en sol ve en sağ adresini gösteren iki işaretçidir (pointer).

- Döngü şartı $i < j$'dir. Dizi tek elemanlıysa ($i = j = 1$) döngüye hiç girilmez; o eleman $x$'e eşitse cevap $i$, değilse 0'dır.
- $i = 1$, $j = 10$ ise orta nokta $m = \lfloor (1 + 10)/2 \rfloor = \lfloor 5.5 \rfloor = 5$'tir. Burada geçen derste görülen taban fonksiyonu kullanılır.
- $x > a_m$ ise aranan sayı $m$'nin sağındadır: $i := m + 1$.
- Değilse ($x \leq a_m$) aranan sayı $m$'de ya da solundadır: $j := m$.
- Döngüden çıkınca $x = a_i$ ise `location` $i$'dir; değilse 0'dır, yani sayı dizide yoktur.

Hoca anlatırken bir an "küçükse $i$ değişir" diye karıştırdı, sonra düzeltti: $x$ küçük ya da eşitse değişen şey sağ uçtur ($j := m$).

Hoca derste dizi üzerinde adım adım izleme yapmadı. Python ile $1, 2, 3, 4, 5, 6, 7, 8, 13$ dizisinde 13 aranınca (ara adım eklendi):

| Adım | $i$ | $j$ | $m$ | $a_m$ | Karar |
|---|---|---|---|---|---|
| 1 | 1 | 9 | 5 | 5 | $13 > 5$, $i := 6$ |
| 2 | 6 | 9 | 7 | 7 | $13 > 7$, $i := 8$ |
| 3 | 8 | 9 | 8 | 8 | $13 > 8$, $i := 9$ |
| son | 9 | 9 | | | $a_9 = 13 = x$, location $= 9$ |

Dizideki her eleman için algoritmanın doğru konumu verdiği Python ile kontrol edildi.

## 19. Sıralama ve Kabarcık Sıralaması

### 19.1 Sıralama Neden Önemli?

Sıralamada amaç, elemanları sayısal, alfabetik ya da dizgi sırasına koymaktır. Gerçek dünyadan bir örnek sözlüktür: Sözlükte "aba" her zaman "ada"dan önce gelir. Kelimeler önce birinci karaktere, sonra ikinci, üçüncü, dördüncü karaktere göre sıralanır. Karışık bir sürü Türkçe kelimeyi sözlüğe sıralı yerleştirmek bir sıralama problemidir.

Slayttaki ifadeye göre donanım kaynaklarının önemli bir yüzdesi, özellikle belirli bir sırayla sunulması gereken büyük veri tabanlarını sıralamaya ayrılır. Hocanın örneği bir bankaydı: O gün en çok hangi işlem türünün (EFT, havale, swift) yapıldığını bulmak için de işlemleri toplayıp sıralamak gerekir.

Çok sayıda sıralama algoritması vardır ve birbirlerine göre avantaj ve dezavantajları vardır. Bunlar veri yapıları ve algoritmalar derslerinde ayrıntılı görülecek.

### 19.2 Kabarcık Sıralaması: Sözde Kod

Kabarcık sıralaması (bubble sort), hocanın deyişiyle "en hımbıl" sıralamalardan biridir.

```
procedure bubble sort(a1, ..., an: real numbers with n >= 2)
for i := 1 to n - 1
    for j := 1 to n - i
        if aj > aj+1 then interchange aj and aj+1
{ a1, ..., an artan sıradadır }
```

Dizi reel sayılardan oluşur ve $n \geq 2$ olmalıdır. İki döngü vardır: $i$, 1'den $n-1$'e; $j$, 1'den $n-i$'ye gider. Her adımda $a_j$ ile $a_{j+1}$ karşılaştırılır. Küçükten büyüğe sıralanıyorsa ve $a_j > a_{j+1}$ ise ikisi yer değiştirir; değilse yerinde kalır.

### 19.3 İndislerin Değişimi

Hoca indislerin nasıl değiştiğini dört elemanlı bir dizi ($n = 4$) üzerinde gösterdi:

- $i = 1$: $j = 1, 2, 3$ ($n - i = 3$)
- $i = 2$: $j = 1, 2$ ($n - i = 2$)
- $i = 3$: $j = 1$ ($n - i = 1$)

İç döngünün sınırı her geçişte bir azalır. Sebebi şudur: Her geçişte kalan en büyük eleman sona yerleşir. İlk geçişten sonra en büyük eleman son gözdedir ve bir daha kontrol edilmesine gerek yoktur. İkinci geçişte ikinci büyük eleman, üçüncüde üçüncü büyük eleman yerleşir. En başta kalan da en küçük eleman olur.

> **Ders sonrası not:** Hoca bir yerde özet olarak "$i = 2$ iken $j = 1, 2, 3$" dedi. Doğrusu yukarıdaki gibidir: $i = 2$ iken $j$ yalnız 1 ve 2 değerlerini alır.

### 19.4 Örnek: $3, 2, 4, 1, 5$

$n = 5$'tir.

$i = 1$ ($j = 1, \ldots, 4$):

- $3 > 2$: yer değiştir. $2, 3, 4, 1, 5$
- $3 < 4$: değişiklik yok.
- $4 > 1$: yer değiştir. $2, 3, 1, 4, 5$
- $4 < 5$: değişiklik yok.
- Sonuç $2, 3, 1, 4, 5$. En büyük eleman 5 sona yerleşti.

$i = 2$ ($j = 1, \ldots, 3$):

- $2 < 3$: değişiklik yok.
- $3 > 1$: yer değiştir. $2, 1, 3, 4, 5$
- $3 < 4$: değişiklik yok.
- Sonuç $2, 1, 3, 4, 5$. 4 de yerleşti.

$i = 3$ ($j = 1, 2$):

- $2 > 1$: yer değiştir. $1, 2, 3, 4, 5$
- $2 < 3$: değişiklik yok.
- 3 de yerleşti.

$i = 4$ ($j = 1$):

- $1 < 2$: değişiklik yok. Dizi $1, 2, 3, 4, 5$.

Dış döngü $i = 5$'e gitmez, çünkü sınır $n - 1$'dir. Hoca bunun "esprisini" şöyle açıkladı: Beşinci geçişte karşılaştırılacak bir eleman kalmaz; en baştaki zaten en küçük elemandır. Sınır $n$ yapılsa bile iç döngüye girilmez, yapılacak bir iş yoktur. Python ile her geçişin sonucu doğrulandı.

Kabarcık sıralaması, bulduğu büyük sayıyı sona atarak ilerleyen bir sıralamadır. Hoca, kabarcık sıralamasını seçmeli sıralamadan (selection sort) ayıran tarafın bu yer değiştirme düzeni olduğunu söyledi.

## 20. Araya Ekleme Sıralaması

### 20.1 Fikir

Araya ekleme sıralaması (insertion sort) ikinci elemandan başlar. İkinci eleman birinciyle karşılaştırılır; daha küçükse birincinin önüne konur. Sonra üçüncü elemana geçilir ve ilk iki elemanla karşılaştırılarak doğru yere yerleştirilir. Bu işlem $n$'inci elemana kadar tekrarlanır. Yani $j$'inci adımda $j$'inci eleman, ilk $j$ eleman arasında doğru yerine "araya eklenir".

### 20.2 Sözde Kod

```
procedure insertion sort(a1, ..., an: real numbers with n >= 2)
for j := 2 to n
    i := 1
    while aj > ai
        i := i + 1
    m := aj
    for k := 0 to j - i - 1      { * }
        a(j-k) := a(j-k-1)
    ai := m
{ a1, ..., an artan sıradadır }
```

Hoca kodu şöyle açıkladı:

- $j$ her zaman ikinci elemandan başlar, bu yüzden $j := 2$.
- $i := 1$'den başlanır ve `while` döngüsüyle $a_j$'nin gireceği yer bulunur: $a_j > a_i$ olduğu sürece $i$ artırılır.
- Yer bulununca $a_j$, `m` değişkeninde saklanır.
- Yıldızlı (*) döngü, $i$ ile $j - 1$ arasındaki elemanları birer göz sağa kaydırır. $a_j$'nin eski gözüne başka bir sayı yazılabilir; değer `m`'de saklandığı için kaybolmaz.
- Son olarak açılan $i$'nci göze saklanan değer konur.

### 20.3 Örnek: $3, 2, 4, 1, 5$

- $j = 2$**:** Eleman 2. $2 < 3$ olduğu için yeri birinci gözdür. 2 saklanır, 3 bir sağa kayar, 2 birinci göze konur: $2, 3, 4, 1, 5$.
- $j = 3$**:** Eleman 4. 4, hem 2'den hem 3'ten büyüktür; yer değiştirme yoktur ve kaydırma döngüsü çalışmaz: $2, 3, 4, 1, 5$.
- $j = 4$**:** Eleman 1. 1, 2'den, 3'ten ve 4'ten küçüktür. 1 saklanır; 4, 3 ve 2 birer göz sağa kayar; 1 birinci göze konur: $1, 2, 3, 4, 5$.
- $j = 5$**:** Eleman 5. Hiçbir elemandan küçük değildir; değişiklik yok: $1, 2, 3, 4, 5$.

Sözde kod Python'a aynen aktarılarak her adım doğrulandı ($j = 2$'de $i = 1$, $j = 3$'te $i = 3$, $j = 4$'te $i = 1$, $j = 5$'te $i = 5$).

> **Ders sonrası not:** Hoca örneği anlatırken yeni elemanı "geriye doğru" karşılaştırdı (örneğin 4'ü önce 3 ile, sonra 2 ile; 1'i önce 4, sonra 3, sonra 2 ile). Slayttaki sözde kod ise yeri dizinin **başından** başlayarak doğrusal aramayla bulur ($i := 1$'den ileri). İki yol da aynı sonucu verir; geriye doğru karşılaştırıp kaydıran sürüm de yaygın bir araya ekleme yazımıdır.
>
> **Hocanın vurgusu:** Bir algoritmayı anlamak için küçük bir örnek üzerinde $i$ ve $j$ indislerinin nasıl değiştiğini adım adım takip edin. Önceden bu konuda sorununuz

yoksa bir şey demem, ama varsa bu şekilde analiz etmek sizin için bulunmaz bir nimettir.

# 21. Öklid Algoritması

## 21.1 En Büyük Ortak Bölen

Öklid algoritması (Euclidean algorithm), iki sayının en büyük ortak bölenini (EBOB, greatest common divisor) bulur. Ortaöğretimde 60 ve 24 gibi iki sayı verildiğinde ikisini de bölünebildiği kadar bölüp çarpanları bulur, en büyük ortak çarpanı seçerdik. Algoritmik olarak düşününce bunun birçok yolu vardır; slaytta iki üç yöntem verilmişti. Hocaya göre en anlamlı ve en optimize edilmiş olanı Öklid algoritmasıdır.

## 21.2 Örnek: $\gcd(60, 24)$

Girişler $m = 60$, $n = 24$'tür.

1. Bir sonraki adımda $m$'nin yerine $n$ yazılır: yeni $m = 24$. $n$'nin yerine de eski $m$'nin eski $n$'ye göre modu yazılır: $60 \bmod 24$. $24 \cdot 2 = 48$ ve $60 - 48 = 12$ olduğu için yeni $n = 12$.
2. Yine $m$'nin yerine $n$ yazılır: $m = 12$. $n$'nin yerine $24 \bmod 12 = 0$ yazılır: $n = 0$.
3. İşlem $n$ sıfır olana kadar sürer. $n = 0$ olduğu için durulur. EBOB, $m$'deki sayıdır: **12**.

| Adım | $m$ | $n$ | $m \bmod n$ |
|---|---|---|---|
| 1 | 60 | 24 | 12 |
| 2 | 24 | 12 | 0 |
| son | 12 | 0 | |

Python'un `math.gcd(60, 24)` sonucu da 12'dir.

## 21.3 Sözde Kod

Hocanın anlattığı biçimiyle:

```
procedure gcd(m, n: positive integers)
while n != 0
    r := m mod n
    m := n
    n := r
return m
```

$n$ sıfırdan farklı olduğu sürece döngü döner. $m$'nin $n$'ye göre modu $r$'ye alınır, $n$'nin değeri $m$'ye, $r$ de $n$'ye atanır. $n$ sıfır olunca döngüden çıkılır ve $m$ döndürülür.

## 21.4 Ardışık Tam Sayı Kontrolü

Slaytta EBOB için ikinci bir algoritma, "ardışık tam sayı kontrolü" algoritması da vardı. Hoca bu algoritmanın sözde kodunu derste anlatmadı. Öğrencilerin onu kendilerinin analiz etmesini ve Öklid ile arasındaki farkı görmesini istedi; Öklid varken onun karışık geleceğini söyledi.

> **Ek bilgi (derste anlatılmadı):** Ardışık tam sayı kontrolü (consecutive integer checking) yönteminde $t$, $m$ ile $n$'nin küçüğünden başlatılır. $t$ hem $m$'yi hem $n$'yi tam bölüyorsa cevap $t$'dir; bölmüyorsa $t$ bir azaltılıp tekrar denenir. Bu yöntem en kötü durumda küçük sayı kadar deneme yapar. Öklid ise her adımda sayıları hızla küçülttüğü için çok daha az adımda biter.

# 22. Algoritma Tasarım Yöntemleri

Slayttaki algoritma tasarım yöntemleri:

- Kaba kuvvet (brute force)

- Böl ve yönet (divide and conquer)
- Azalt ve yönet (decrease and conquer)
- Dönüştür ve yönet (transform and conquer)
- Bellek-zaman ödünleşimi (space and time trade-off)
- Açgözlü algoritma (greedy algorithm)
- Dinamik programlama (dynamic programming)
- Yinelemeli iyileştirme (iterative improvement)
- Geri izleme (backtracking)
- Dal ve sınır (branch and bound)

Her birinin örnekleri algoritma analizi dersinde görülecek. Hoca dinamik programlamanın bence en güzel tasarım yöntemlerinden biri olduğunu, ama dinamik programlama yazmanın kolay olmadığını söyledi. Derste görülen sıralamalar biraz kaba kuvvete girer. Hızlı sıralama (quick sort) ise bir böl ve yönet algoritmasıdır.

Önümüzdeki iki haftada bir algoritmanın analizinin nasıl yapıldığı görülecek: yürütme zamanı nasıl hesaplanır ve yürütme zamanından algoritmanın karmaşıklığına nasıl geçilir. Algoritmalar ve karmaşıklıkları birer fonksiyon olarak ele alınıp analiz edilecek. Ama ondan önce biraz veri yapılarından söz etmek gerekiyor.

## Örnek Sorular

Bu derste hazır kare yoktur. Sorular, derste hocanın sınıfa sözlü sorduğu ya da öğrencilerin sorduğu sorulardan alınmıştır.

**Soru 1** (derste sözlü, 00:36:16): Slayttaki ok diyagramlarının her biri için: Bu bir fonksiyon mu? Birebir mi? Örten mi?

**Çözüm:** Önce fonksiyon şartına bakılır: tanım kümesindeki her eleman tam olarak bir elemana gitmeli. Fonksiyon değilse diğer özelliklere bakılmaz. Birebir için iki elemanın aynı değere gitmemesi, örten için değer kümesinde açıkta eleman kalmaması gerekir. Diyagramların cevapları Bölüm 4.1'deki tablodadır.

**Soru 2** (derste sözlü, 00:59:28): $\lfloor 8.3 \rfloor$, $\lfloor 1/2 \rfloor$, $\lfloor -8.7 \rfloor$, $\lfloor 7 \rfloor$, $\lfloor -1/2 \rfloor$, $\lfloor 3.1 \rfloor$, $\lceil -11.3 \rceil$, $\lceil 6 \rceil$, $\lceil -8 \rceil$, $\lceil 1/2 \rceil$, $\lceil -1/2 \rceil$, $\lceil 3.1 \rceil$ değerlerini bulun.

**Çözüm:** Sırasıyla $8, 0, -9, 7, -1, 3, -11, 6, -8, 1, 0, 4$. Açıklamalar Bölüm 8.4'teki tablodadır.

**Soru 3** (derste sözlü, öğrenci sorusu, 01:04:00): $\lfloor 1.\overline{9} \rfloor$ nedir? Devirli sayıda taban 1 mi olur, 2 mi?

**Cevap (derste):** Hoca devirli sayının programda nasıl verileceğini sordu. Bir reel değişkene yazılabilecek 9 sayısının değişkenin kapladığı alanla sınırlı olduğunu söyledi. Matematikte pratikte bunun 2 alınması gerektiğini, ama koda sonlu sayıda 9 ile verilen bir sayının tabanının 1 çıkacağını düşündüğünü belirtti.

> **Çözüm (derste verilmedi):** Matematikte $1.\overline{9} = 2$'dir, çünkü $\sum_{k=1}^{\infty} 9 \cdot 10^{-k} = 1$'dir. Bu yüzden $\lfloor 1.\overline{9} \rfloor = 2$'dir. Programda durum, sayının kaç basamakla yazıldığına bağlıdır. Python'da `math.floor(1.999999999999999)` (15 tane 9) sonucu 1'dir. Ama `1.9999999999999999` (16 tane 9) kayan noktalı sayıya çevrilirken zaten `2.0`'a yuvarlanır ve tabanı 2 çıkar. Kısacası bilgisayar sonsuz 9'u saklayamaz; sonuç yazılan basamak sayısına göre 1 ya da 2 olabilir.

**Soru 4** (derste sözlü, öğrenci sorusu, 01:08:07): Biri üstel, biri logaritmik iki fonksiyonla denedim, ama $f \circ g$ ile $g \circ f$ eşit çıkmadı. Değişme özelliği yalnız üstel-logaritmik fonksiyonlarda mı vardır?

**Cevap (derste):** Hoca üstel ve logaritmik fonksiyonların birbirinin tersi olduğunun kesin olduğunu, ama $f \circ g = g \circ f$ konusunu kitaptan kontrol edeceğini söyledi.

> **Çözüm (derste verilmedi):** Eşitlik yalnız iki fonksiyon birbirinin tersiyse kesin olarak sağlanır; bunun için tabanlar aynı olmalıdır. $f(x) = 2^x$ ve $g(x) = \log_3 x$ için $f(g(x)) = 2^{\log_3 x} = x^{\log_3 2}$, $g(f(x)) = \log_3 2^x = x \log_3 2$'dir. Örneğin $x = 9$'da birincisi $2^2 = 4$, ikincisi $\log_3 512 \approx 5.68$'dir (Python ile hesaplandı). İkisi eşit değildir.

**Soru 5** (derste sözlü, 01:30:25): $U = \{1, \dots, 10\}$, $A = \{1, 4, 7, 10\}$, $B = \{1, 2, 3, 4, 5\}$, $C = \{2, 4, 6, 8\}$ için $\overline{B} \cap (C - A)$ ve $(A \cup B) - (C - B)$ nedir?

**Çözüm:** $\overline{B} \cap (C - A) = \{6, 7, 8, 9, 10\} \cap \{2, 6, 8\} = \{6, 8\}$. $(A \cup B) - (C - B) = \{1, 2, 3, 4, 5, 7, 10\} - \{6, 8\} = \{1, 2, 3, 4, 5, 7, 10\}$. Ayrıntı Bölüm 9.1'dedir.

**Soru 6** (derste sözlü, 01:35:03): $\Big((C \cap A) - \overline{(B - A)}\Big) \cap C$ ifadesini Venn şemasında gölgelendirin.

**Çözüm:** $C \cap A$ tamamen $A$'nın içindedir ve $A$, $\overline{(B - A)}$'nın içindedir. Fark boş kalır; boş kümenin $C$ ile kesişimi de boştur. Sonuç $\emptyset$ (Bölüm 9.2).

**Soru 7** (derste sözlü, 01:39:47): 191 öğrenciden 65'i Fransızca, 76'sı işletme, 63'ü müzik alıyor. 36'sı F ve B, 20'si F ve M, 18'i B ve M, 10'u üçünü birden alıyor. Kaç kişi Fransızca ve müzik alıp işletme almıyor?

**Çözüm:** $20 - 10 = 10$ kişi. Venn şemasının bütün bölgeleri Bölüm 10'dadır.

**Soru 8** (derste sözlü, 01:44:18): Her $X, Y \subseteq U$ için $(X \cap Y) \cup (Y - X) = X$ doğru mu? Yanlışsa karşı örnek verin.

**Cevap (derste):** Yanlış; sol taraf $Y$'ye eşittir. Slayttaki karşı örnek altyazıda okunmuyor.

> **Çözüm (derste verilmedi):** Karşı örnek: $X = \{1\}, Y = \{2\}$. $X \cap Y = \emptyset$ ve $Y - X = \{2\}$ olduğu için sol taraf $\{2\}$'dir; bu da $X = \{1\}$'e eşit değildir. Python ile küçük kümelerin bütün çiftleri için sol tarafın her zaman $Y$'ye eşit olduğu da kontrol edildi.

**Soru 9** (derste sözlü, 01:53:01): $\{1, \dots, 5\}$ üzerinde $x + y \leq 6$ bağıntısının elemanlarını yazın. Yansımalı, simetrik, ters simetrik, geçişli midir?

**Çözüm:** 15 çift vardır. Yansımalı değil (non-reflexive), simetrik, ters simetrik değil, geçişli değil ($(5, 1)$ ve $(1, 2)$ var, $(5, 2)$ yok). Ayrıntı Bölüm 13.1'dedir.

**Soru 10** (derste sözlü, 02:02:00): Birebir $f(x) = 3 + \dfrac{1}{x}$ fonksiyonunun tersi nedir?

**Çözüm:** $y = 3 + \dfrac{1}{x} \Rightarrow x = \dfrac{1}{y - 3}$, yani $f^{-1}(x) = \dfrac{1}{x - 3}$ (Bölüm 13.4).

**Soru 11** (derste sözlü, 02:24:56): Doğrusal aramada `while` yerine ne kullanılabilirdi? Neden `while` tercih edildi?

**Çözüm:** `for` döngüsü ve içinde bir `if` kullanılabilirdi. Ama sayı erken bulunsa bile `for` sonuna kadar döner ya da `exit`/`goto` gerekir. `while` ise sayı bulununca ya da dizinin sonuna gelinince kendiliğinden biter; bu yüzden daha etkilidir (Bölüm 17.5).

## Dersin Toparlaması

Bu derste önce geçen haftadan kalan Warshall algoritması tekrarlandı. Bağıntı matrise çevrildi ve $w_{ij} := w_{ij} \vee (w_{ik} \wedge w_{kj})$ işleminin, gözle yapılan geçişlilik kontrolünü nasıl tek satırda yaptığı gösterildi. Ardından fonksiyonlar tamamlandı. Fonksiyon şartı, birebir, örten ve birebir-örten fonksiyonlar, liste, doğrusal, kesirli ve polinom biçimlerinde ters fonksiyon, bileşkenin birleşme özelliği ve değişme özelliğinin olmayışı, üstel ve logaritmik fonksiyonlar ile taban ve tavan fonksiyonları işlendi. Bölümün soru-cevap slaytlarında küme işlemleri, Venn şeması, üç kümeli sayma, karşı örnek, dizi toplamları, sayı tabanları ve bağıntı özellikleri soruları birlikte çözüldü.

Dersin ikinci yarısında yeni algoritmalar bölümüne geçildi. Algoritmanın tanımı ve özellikleri, sözde kodun neden kullanıldığı, en büyük elemanı bulma, doğrusal ve ikili arama, kabarcık ve araya ekleme sıralaması ile Öklid algoritması görüldü. Hoca her algoritmayı küçük bir örnek üzerinde indisleri takip ederek anlamayı ve uç durumları düşünmeyi önerdi. Son olarak algoritma tasarım yöntemleri listelendi. Önümüzdeki iki haftada yürütme zamanı ve karmaşıklık analizi işlenecek; öncesinde temel veri yapılarına kısaca değinilecek.

**Not:** İsterseniz bu kayıtları Gemini Notebook gibi bir araca atıp daha istifadeli hale getirebilirsiniz.